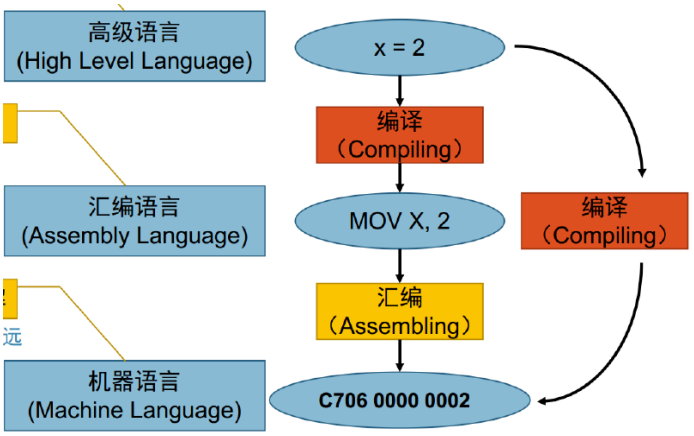
第一章 引论

> Important
>
> **重点：**
>
> - 编译器划分
> - 各阶段的主要功能
> - 符号表在各阶段的使用

基本概念

1、编译

将**高级语言**翻译成**汇编语言**或**机器语言**的过程



2、源语言 & 目标语言

- 源程序：被翻译的初始语言
- 目标语言：翻译后生成的语言

3、翻译器 & 编译器 & 解释器

- 翻译器：将源语言翻译成目标语言，总称
- 编译器：对源程序进行完整的分析，并生成可执行文件（目标程序）

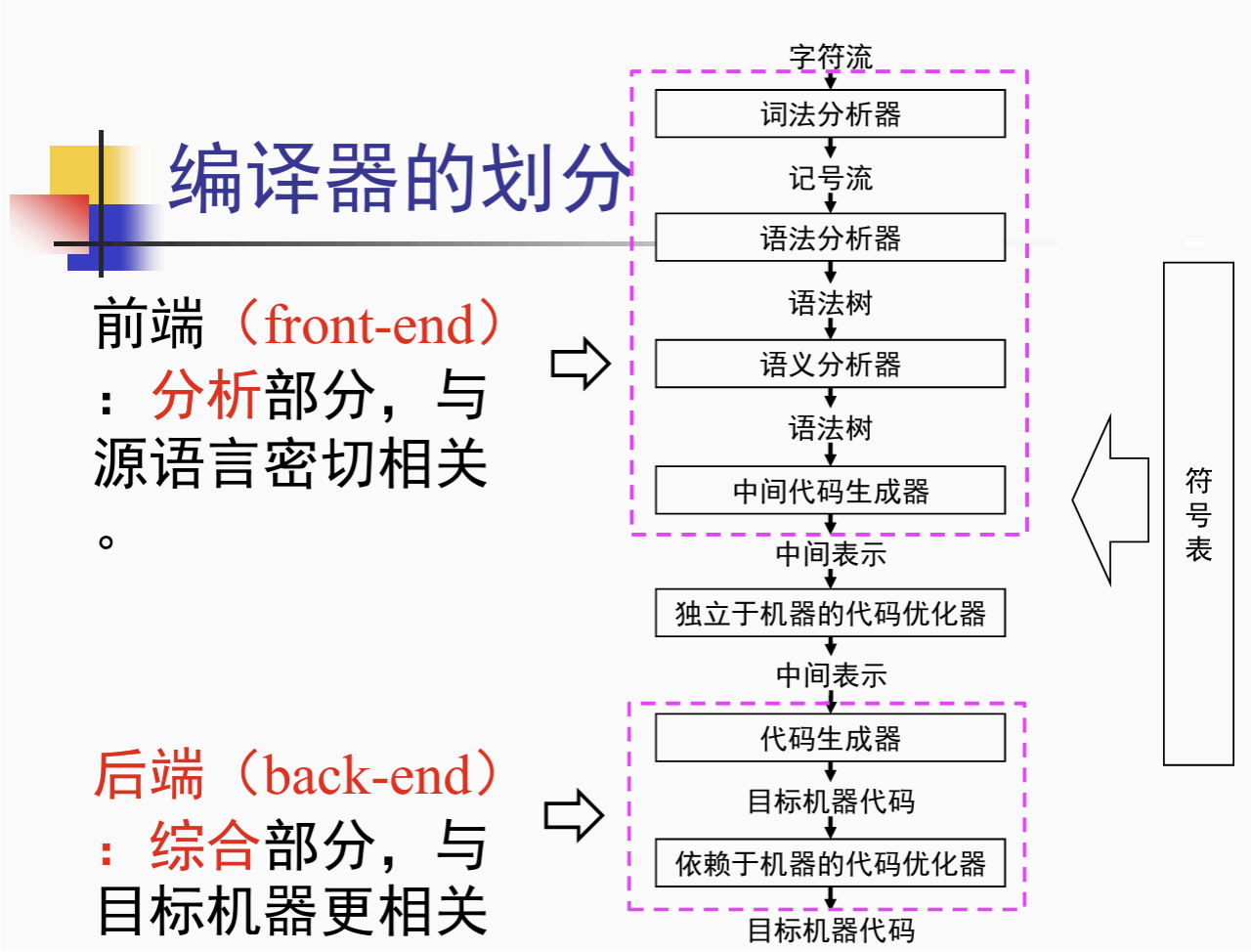
  👉 **一次翻译，多次执行**
- 解释器：在运行时读取源程序，逐行进行分析翻译，并立即执行翻译后的动作

  👉 **边翻译，边执行**

4、交叉编译

在一个平台上生成另一个平台上的可执行代码，**编译环境 ≠ 运行环境**

编译器的划分



编译程序的组织

1、遍

指编译器对源程序（或其等价的中间表示）进行**从头到尾的一次完整扫描和处理**

👉 现在的编译器（如 GCC, LLVM）通常是**多遍扫描**的。例如，它们会先扫描整个程序构建语法树，然后再扫描语法树进行类型检查，最后再扫描中间代码进行优化。

2、编译阶段分组

- 前端

  词法分析器 + 语法分析器 + 语义分析器 + 中间代码生成器

  分析部分，负责检查代码是否符合该语言的语法和语义规则

- 后端

  代码生成器 + 依赖于机器的代码优化器

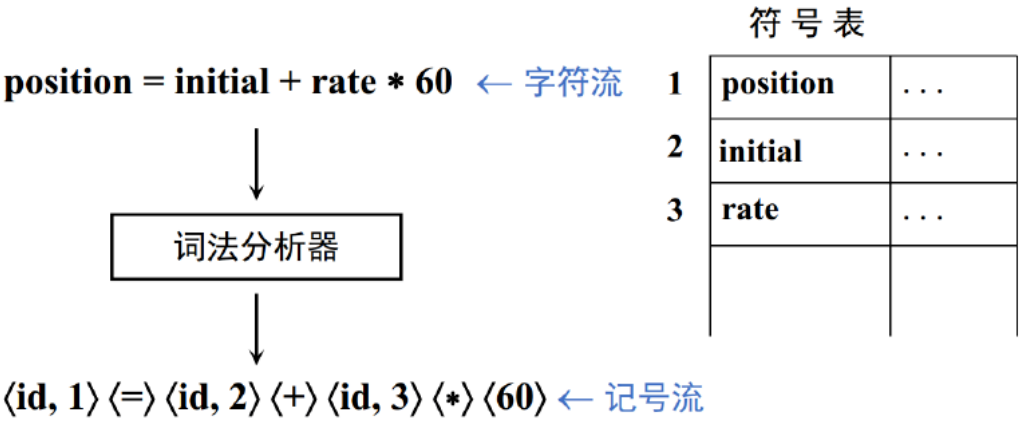
  综合部分，负责将中间表示转换为特定机器的指令。

👉 将编译器拆分为源语言模块 + 目标机器模块，当要开发新语言只需要写响应的前端；需要让语言支持不同的平台只需要针对平台写不同的后端

各阶段主要功能

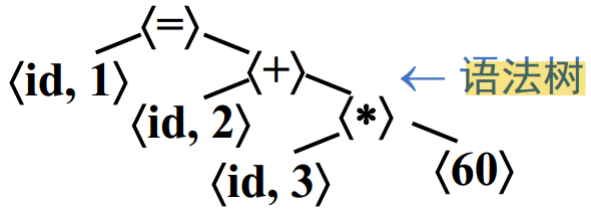
词法分析

- 从**左向右逐行扫描**源程序的字符，识别出各个单词（token），确定单词类型（关键字、标识符、运算符、常量、界限符）
- 输入：字符流（源程序文本）
- 输出：记号流（tokens）



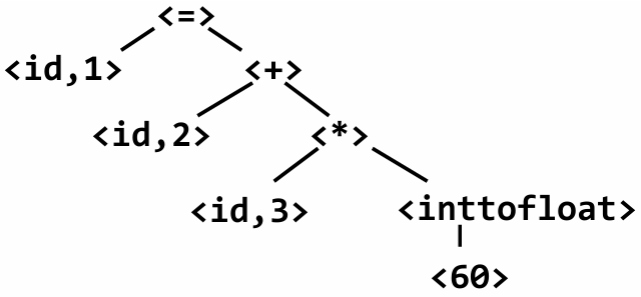
语法分析

- 检查记号流是否满足编程语言的**语法规则**，并将线性排列的单词转换为一颗**层次化的树**，体现代码的嵌套结构。
- 输入：记号流（tokens）
- 输出：语法树（AST）



语义分析

- 分析语法结构的含义和功能；类型检查、语义一致性检查；为代码生成阶段收集信息
- 输入 = 输出 = 语法树



中间代码生成

- 生成一种与机器无关的、类似汇编的通用代码。它既不依赖特定的源语言，也不直接对应特定的 CPU 指令，便于进行**与平台无关的优化**。
- 输入：语法树

- 输出：中间表示（逆波兰表示/后缀表示、三地址码、树型结构

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

特点：
- 必须先决定运算次序
- 临时变量
- 最多有三个运算对象

独立于机器的代码优化

- 进行**等价程序变换**，产生更好的目标代码
- 输入 = 输出 = 中间表示
- 目标：运行更快、代码更短、能耗更低

```
t1 = id3 * 60.0
id1 = id2 + t1
```

代码生成

- 分配寄存器或内存单元，将中间代码翻译成等价的机器指令序列
- 输入：优化后的中间代码。
- 输出：目标机器代码（汇编代码或二进制指令）。

```
MOVF id3,  R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

依赖于机器的优化

- 针对**特定目标机器**的代码优化
- **输入**：目标机器代码。
- **输出**：更高效的目标机器代码。

符号表

编译器中最重要的数据结构之一，“核心档案室”或“字典”。

记录了程序中所有的**标识符**（如变量名、函数名、类名、类型名等）以及这些标识符的各种**属性**（如数据类型、作用域、存储位置、偏移量等）。

各阶段使用方式

词法分析阶段（创建）

- **功能**：当词法分析器识别出一个标识符（ID）时，它会首先查询符号表。
- **操作**：
  - 如果该标识符不存在，将其插入符号表。
  - 如果是关键字（如 if），通常直接返回预定义的 Token 类型，不一定放入符号表。

语法分析阶段（填充）

- **功能**：配合语法分析过程，处理作用域（Scope）。
- **操作**：
  - 当进入一个代码块（如 {）时，符号表需要开启一个新的**作用域层级**。
  - 当离开代码块（如 }）时，需要销毁或隐藏当前层级的符号。
  - 此时开始填充标识符的基本声明信息（如 int a; 会把 a 的类型信息录入）。

语义分析阶段（查询与验证 —— 最核心阶段）

- **功能**：这是符号表被使用最频繁的阶段，用于保证程序的逻辑正确性。
- **操作**：
  - **声明检查**：使用变量前是否已经在符号表中定义？（报"未定义变量"错误）。
  - **类型检查**：在进行 a + b 操作时，去符号表查 a 和 b 的类型。如果 a 是 int，b 是 string，编译器通过符号表就能发现类型不匹配，报"类型错误"。
  - **作用域验证**：确认当前访问的变量在当前作用域内是否可见。

中间代码生成阶段（记录位置信息）

- **功能**：记录变量在内存中的分配方案。
- **操作**：编译器在生成中间代码时，需要确定变量存在哪里。符号表会记录每个变量相对于当前活动记录（栈帧）的**偏移量**。例如：a 位于栈顶向下 4 个字节处。

代码生成阶段（最终应用）

- **功能**：利用符号表中的地址信息生成机器指令。
- **操作**：当目标代码生成器需要生成汇编指令（如 MOV EAX, [EBP-8]）时，它会查符号表，取出变量 a 对应的存储位置信息（EBP-8），将其填入机器指令中。